

Instituto Tecnológico y de Estudios Superiores de Monterrey

Escuela de Ingeniería y Ciencias

Ingeniería en Ciencias de Datos y Matemáticas

Reporte Técnico

Uso de Álgebras Modernas para Seguridad y Criptografía

Integrantes del equipo

Carola Vázquez Arjona	A01754562
Iker Álvarez Bandrés	A01282986
Juan Pablo Valdés Cárdenas	A00839013
Andrés Vivanco Treviño	A01723358
Juan Fernando González	A01571586
Carlos Eduardo Ramos Martínez	A01286025

Equipo 4 | Grupo 604

Profesores

Raúl Gómez, Anas Wajid y Alberto F. Martínez

Socio Formador

Casa Monarca, Ayuda Humanitaria al Migrante, A.B.P.

Monterrey, Nuevo León

12 de junio de 2026

Índice

Abstract	1
1. Introducción	2
2. Marco de Referencia	3
2.1. Criptografía de clave pública	3
2.2. Funciones hash criptográficas	3
2.3. Certificados digitales y X.509	4
2.4. Firma de correos electrónicos: S/MIME	4
2.5. Control de acceso basado en roles (RBAC)	5
2.6. Bibliotecas criptográficas evaluadas	5
2.7. Sistemas comparables	6
2.8. Estándares de cifrado en reposo	6
2.9. Marco legal aplicable	7
3. Desarrollo	7
3.1. Propuesta	7
3.1.1. Visión general de la solución	7
3.1.2. Justificación de la elección de stack tecnológico	7
3.1.3. Justificación de HostGator frente a AWS u otras plataformas cloud	8
3.1.4. Justificación del modelo zero-knowledge para llaves privadas	9
3.1.5. Justificación de la decisión de firma S/MIME con clave del sistema	9
3.1.6. Justificación del modelo de comunicación externa diferenciado	9
3.1.7. Justificación de la validación en tres capas para adjuntos	10
3.1.8. Diagrama de la solución	11
3.2. Desarrollo de componentes: Backend	12
3.2.1. Router único y despacho de peticiones	12
3.2.2. Middleware RBAC	12
3.2.3. Controlador de autenticación	12
3.2.4. Controlador de certificados	12
3.2.5. Controlador de documentos	13
3.2.6. Helper de mensajería	13
3.2.7. Helper de validación de archivos	13
3.2.8. Mailer con firma S/MIME	13
3.2.9. Tareas programadas	14

3.2.10.	Bitácora institucional	14
3.3.	Desarrollo de componentes — Frontend	14
3.3.1.	Frontend principal: frontend.html	14
3.3.2.	Sidebar dinámico	15
3.3.3.	Wrapper de API y manejo de tokens	15
3.3.4.	Firma digital local con Web Crypto API	15
3.3.5.	Modales reutilizables	16
3.3.6.	Páginas públicas dedicadas	16
3.3.7.	Componentes visuales y feedback al usuario	17
3.3.8.	Visualización de permisos con iconos	17
3.4.	Conexión de los componentes	17
3.4.1.	Comunicación entre frontend y backend	17
3.4.2.	Autenticación y manejo de sesiones	18
3.4.3.	Comunicación entre backend y base de datos	18
3.4.4.	Integración con el subsistema criptográfico del navegador	18
3.4.5.	Integración con el servidor SMTP para correos firmados	19
3.4.6.	Tareas programadas mediante cron	19
3.4.7.	Flujo completo de un caso ilustrativo	20
4.	Resultados	21
4.1.	Resultados del Backend	21
4.2.	Resultados del Frontend	21
4.3.	Resultados de la solución completa	22
4.4.	Casos de uso y tabla de validaciones	22
5.	Conclusiones y trabajo futuro	25
5.1.	Conclusiones	25
5.2.	Trabajo futuro	25
Anexo:	Repositorio del proyecto	26

Índice de tablas

1.	Casos de uso y tabla de validaciones del sistema SIGAM. ¹	23
----	--	----

Índice de figuras

1.	Diagramas de la propuesta de solución. ²	11
----	---	----

Abstract

El presente reporte documenta el desarrollo del Sistema Institucional de Gestión para Casa Monarca (SIGAM), una plataforma web diseñada para resolver la ausencia de mecanismos seguros de gestión de identidades, control de accesos y protección documental en una organización dedicada a la asistencia humanitaria de personas migrantes. La solución integra cinco módulos operativos —gestión de identidades, infraestructura de clave pública (PKI), gestión de documentos, comunicación interna y comunicación externa— aplicando esquemas criptográficos estándar de la industria: RSA-2048 con SHA-256 para firmas digitales, bcrypt para hashing de contraseñas, X.509 para certificados, S/MIME para correos electrónicos y JWT para autenticación de sesiones. El sistema implementa un principio de zero-knowledge sobre las llaves privadas de los usuarios, garantizando que estas nunca se almacenan en el servidor. Cada operación sensible queda registrada en una bitácora institucional inmutable. El resultado es una plataforma completamente funcional desplegable tanto en contenedores Docker locales como en hosting compartido de producción, distribuida bajo licencia MIT para permitir su reutilización por otras organizaciones de la sociedad civil con necesidades similares.

Palabras clave: criptografía aplicada, infraestructura de clave pública, firma digital, RSA, S/MIME, control de acceso basado en roles, zero-knowledge, gestión documental.

1. Introducción

La transformación digital de las organizaciones de la sociedad civil enfrenta un desafío particular: la mayoría de estas instituciones manejan información altamente sensible de poblaciones vulnerables, pero carecen de los recursos técnicos y financieros para implementar las soluciones de seguridad informática que operan grandes empresas o instituciones gubernamentales. Casa Monarca, Ayuda Humanitaria al Migrante, A.B.P., es una organización mexicana dedicada al acompañamiento de personas migrantes y solicitantes de asilo, que en su operación diaria gestiona expedientes, documentos legales, registros médicos y comunicaciones que contienen información personal protegida por la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP).

A inicios del proyecto, Casa Monarca había sufrido un incidente de seguridad que comprometió información personal de su población atendida. El diagnóstico inicial identificó las causas estructurales del problema: la organización no contaba con un sistema centralizado de gestión de identidades digitales, los accesos a la información sensible no estaban diferenciados por rol o área operativa, no existían mecanismos criptográficos para garantizar la autenticidad o integridad de los documentos institucionales, y la comunicación con colaboradores externos se realizaba a través de canales sin garantías técnicas de no repudio.

El reto planteado por el socio formador consistió en diseñar e implementar un sistema institucional que resolviera estos problemas aplicando esquemas criptográficos estándar de la industria, manteniendo la simplicidad operativa para que pudiera ser adoptado por personal sin formación técnica avanzada, y respetando las restricciones presupuestarias de una organización sin fines de lucro. Esto implicó decisiones de arquitectura específicas, como la elección de hosting compartido en lugar de infraestructura cloud especializada, el uso de tecnologías ampliamente soportadas (PHP, MySQL, Apache) en lugar de stacks más modernos pero menos accesibles, y la adopción del navegador del usuario como plataforma criptográfica para preservar el principio de zero-knowledge sobre las llaves privadas.

El sistema desarrollado, denominado SIGAM (Sistema Institucional de Gestión para Casa Monarca), se construyó en dos etapas a lo largo del semestre. La primera etapa estableció la infraestructura básica de gestión de identidades y la infraestructura de clave pública (PKI). La segunda etapa amplió el sistema con los módulos de gestión documental, comunicación interna firmada, comunicación externa con verificación criptográfica de adjuntos y firma S/MIME en correos institucionales. El resultado final es una plataforma completamente funcional, documentada y lista para despliegue en producción.

Este reporte documenta de manera técnica las decisiones de diseño tomadas, la arquitectura implementada, los algoritmos criptográficos aplicados, los casos de uso validados y las

áreas de mejora identificadas para una eventual versión 2 del sistema. La estructura del documento sigue el formato de reporte técnico estándar, comenzando con el marco teórico que respalda las decisiones criptográficas, continuando con el desarrollo detallado de la propuesta y sus componentes, y cerrando con la presentación de resultados, el impacto de negocio cuantificado y las conclusiones del proyecto.

2. Marco de Referencia

Esta sección presenta los fundamentos teóricos y las soluciones existentes que se revisaron durante el diseño del sistema SIGAM. La revisión incluyó esquemas criptográficos estándar, bibliotecas de uso común para implementar dichos esquemas, sistemas comparables operados por instituciones públicas y privadas, y los estándares formales que sustentan las decisiones técnicas tomadas.

2.1. Criptografía de clave pública

La criptografía asimétrica o de clave pública es la base sobre la que se construyen las garantías de autenticidad, integridad y no repudio en sistemas distribuidos modernos. A diferencia de la criptografía simétrica, donde la misma clave se usa para cifrar y descifrar, los esquemas asimétricos operan con un par de claves matemáticamente vinculadas: una clave pública que puede distribuirse libremente y una clave privada que únicamente conserva su propietario. Lo que se cifra con una solo puede descifrarse con la otra, propiedad que permite construir esquemas de firma digital, intercambio seguro de claves y cifrado autenticado [1].

El algoritmo RSA, propuesto por Rivest, Shamir y Adleman en 1978, sigue siendo el esquema asimétrico más ampliamente adoptado a pesar del surgimiento de alternativas como criptografía de curva elíptica. RSA basa su seguridad en la dificultad computacional de factorizar números compuestos grandes; con tamaños de clave de 2048 bits o superiores, se considera resistente contra ataques con la tecnología computacional actual y para los próximos años [2]. Para este proyecto se eligió RSA-2048 por su amplio soporte en bibliotecas estándar, su compatibilidad con certificados X.509 y la abundante documentación disponible.

2.2. Funciones hash criptográficas

Las funciones hash criptográficas mapean entradas de tamaño arbitrario a salidas de tamaño fijo con propiedades específicas: resistencia a colisiones, resistencia a preimagen y efecto avalancha. SHA-256, definida en el estándar FIPS 180-4 del NIST [3], produce un resumen de 256 bits y se utiliza en el proyecto en tres contextos: como función hash dentro

de las firmas RSA (RSASSA-PKCS1-v1_5 con SHA-256), como mecanismo de verificación de integridad de archivos adjuntos almacenados en la base de datos, y como componente del algoritmo HMAC-SHA-256 utilizado por los tokens JWT.

Para el almacenamiento de contraseñas se evaluaron alternativas como SHA-256 con salt, PBKDF2, `srypt` y `bcrypt`. Se eligió `bcrypt` por su factor de coste configurable que permite aumentar la dificultad computacional conforme avance el hardware disponible, manteniéndose resistente a ataques de fuerza bruta con GPUs modernas [4]. La función `password_hash` nativa de PHP implementa `bcrypt` por defecto con un factor de coste apropiado.

2.3. Certificados digitales y X.509

El estándar X.509, definido por la ITU-T y mantenido por el IETF, especifica el formato de los certificados digitales utilizados por la mayoría de los sistemas de seguridad en internet, incluyendo TLS/SSL, S/MIME y firma de código [5]. Un certificado X.509 vincula una clave pública con una identidad (persona, organización o servicio) mediante la firma de una autoridad certificadora confiable. Para SIGAM, Casa Monarca actúa como su propia CA interna, emitiendo certificados a sus colaboradores que únicamente tienen validez dentro del contexto institucional. Esta decisión es consciente: implementar certificados emitidos por una CA pública reconocida (como Let's Encrypt o DigiCert) habría requerido recursos adicionales sin aportar valor proporcional, dado que los certificados se utilizan exclusivamente para comunicación interna y los destinatarios externos pueden verificar firmas mediante el verificador público integrado en el sistema.

2.4. Firma de correos electrónicos: S/MIME

S/MIME (Secure/Multipurpose Internet Mail Extensions) es el estándar más extendido para firma y cifrado de correos electrónicos. Su versión 3.2, definida en el RFC 5751 [6], especifica cómo aplicar firmas digitales sobre mensajes MIME multipart. La firma se calcula sobre el contenido completo del mensaje, incluyendo sus headers principales y todos los body parts contenidos. Esta propiedad es central para SIGAM: significa que una sola firma S/MIME aplicada al correo cubre tanto el cuerpo del mensaje como cualquier archivo adjunto incluido, sin requerir firmas adicionales por adjunto. El RFC 5652 [7], que define la sintaxis criptográfica subyacente (Cryptographic Message Syntax, anteriormente PKCS#7), respalda formalmente esta cobertura.

2.5. Control de acceso basado en roles (RBAC)

El modelo de control de acceso basado en roles (RBAC) fue formalizado por Sandhu et al. en la década de 1990 y posteriormente estandarizado por el NIST [8]. RBAC asigna permisos a roles en lugar de directamente a usuarios, y los usuarios heredan los permisos del rol al que pertenecen. Para SIGAM se adoptó una variante extendida del modelo RBAC básico: además de los cuatro niveles jerárquicos (Administrador, Coordinador, Operativo, Consulta), se incorporó la dimensión de área operativa (Humanitaria, PsicoSocial, Legal, Comunicación, Almacén, TI) y un mecanismo de overrides individuales que permite ajustar permisos para casos específicos sin necesidad de promover al usuario a un nivel superior.

2.6. Bibliotecas criptográficas evaluadas

Durante el diseño se revisaron varias bibliotecas criptográficas como referencia para implementar las primitivas requeridas:

Pycryptodome es una biblioteca de Python ampliamente utilizada que implementa esquemas asimétricos (RSA, DSA, ECDSA), funciones hash, cifrado simétrico (AES, ChaCha20) y construcciones de alto nivel como PKCS#7. Se evaluó como referencia conceptual para entender la API esperada por desarrolladores, aunque finalmente no se utilizó porque el proyecto se implementó en PHP. La consistencia entre la API de Pycryptodome y la extensión OpenSSL de PHP es alta, lo que permitió aplicar patrones de uso conocidos sin curva de aprendizaje adicional.

OpenSSL es la biblioteca criptográfica de facto en sistemas Unix y se expone en PHP mediante la extensión nativa con el mismo nombre. SIGAM utiliza esta biblioteca para todas las operaciones del backend: generación de pares de claves (`openssl_pkey_new`), emisión de certificados (`openssl_csr_sign`), verificación de firmas (`openssl_verify`), aplicación de firmas S/MIME (`openssl_pkcs7_sign`) y verificación de las mismas (`openssl_pkcs7_verify`). La robustez y madurez de OpenSSL la hacen apropiada para sistemas en producción.

Web Crypto API es el estándar W3C que expone primitivas criptográficas dentro de los navegadores web modernos. SIGAM utiliza esta API para implementar el principio de zero-knowledge sobre las llaves privadas de los usuarios: las operaciones de firma se ejecutan localmente en el navegador del cliente mediante `window.crypto.subtle.sign`, sin que la clave privada se transmita al servidor en ningún momento. Esta capacidad es relativamente reciente y diferencia a SIGAM de soluciones tradicionales donde el servidor maneja las claves de los usuarios.

2.7. Sistemas comparables

Se revisaron sistemas en producción que abordan problemáticas similares a las de Casa Monarca, con el objetivo de identificar patrones de diseño aplicables:

Sistema del Servicio de Administración Tributaria (SAT) de México. El SAT implementa un esquema de firma electrónica avanzada conocido como e.firma, basado en certificados X.509 emitidos por una autoridad certificadora pública oficial. Los contribuyentes utilizan su llave privada para firmar declaraciones fiscales y otros trámites, garantizando no repudio jurídicamente vinculante. SIGAM se inspira en este modelo, particularmente en la separación entre la llave privada del usuario (resguardada localmente) y el certificado público (almacenado por la institución para fines de verificación). La diferencia fundamental es que el SAT opera con una CA pública reconocida internacionalmente, mientras que Casa Monarca opera con una CA interna apropiada para su contexto institucional.

Microsoft Entra ID (anteriormente Azure Active Directory). La plataforma de gestión de identidades empresariales de Microsoft [9] ofrece capacidades extensas: federación de identidades, autenticación multifactor, control de acceso condicional, single sign-on con miles de aplicaciones. Su modelo de roles y permisos sirvió como referencia para diseñar el RBAC de SIGAM. La principal diferencia es de escala y propósito: Entra ID es una solución empresarial generalista, mientras que SIGAM es una aplicación específica para Casa Monarca, donde la simplicidad operativa y la auto-suficiencia institucional son requisitos críticos.

Azure Key Vault y servicios equivalentes en otros proveedores cloud [10] ofrecen almacenamiento gestionado de secretos criptográficos, incluyendo claves privadas. Se evaluaron como alternativa para el manejo de la clave del sistema utilizada en la firma S/MIME, pero se descartaron por dos razones: el costo recurrente excede el presupuesto del proyecto, y agregaría una dependencia externa que contradice el principio de auto-suficiencia institucional buscado por el socio formador. En su lugar, la clave del sistema se almacena en el filesystem del servidor con permisos restringidos al usuario de Apache.

2.8. Estándares de cifrado en reposo

Como referencia para la mejora identificada de cifrado en reposo de la base de datos, se revisó el estándar Transparent Data Encryption (TDE) que ofrecen tanto SQL Server [11] como MySQL. TDE cifra los archivos físicos del almacenamiento mediante AES-256, conforme al estándar FIPS 197 [12], de manera transparente al código de la aplicación. La clave maestra del cifrado se gestiona mediante un keyring separado, de forma que un dump de la base de datos sin acceso simultáneo al keyring resulta ilegible. Esta capacidad queda planteada como prioridad para la versión 2 del sistema.

2.9. Marco legal aplicable

La operación de Casa Monarca, al manejar datos personales sensibles de personas en situación de vulnerabilidad, queda sujeta a la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) y su reglamento. El diseño de SIGAM considera principios derivados de esta normativa: minimización de datos (recolectar únicamente lo necesario), control de acceso por necesidad de saber (RBAC), trazabilidad de operaciones (bitácora institucional), y respaldo técnico de los derechos ARCO (Acceso, Rectificación, Cancelación, Oposición) que las personas titulares pueden ejercer sobre sus datos. La implementación completa de los mecanismos formales de ARCO queda identificada como mejora para la versión 2.

3. Desarrollo

3.1. Propuesta

La propuesta de solución consistió en desarrollar SIGAM, una plataforma web institucional integral que cubre los flujos críticos de Casa Monarca aplicando criptografía estándar de la industria. Esta sección describe la solución y justifica las decisiones de diseño tomadas durante el desarrollo.

3.1.1. Visión general de la solución

SIGAM integra cinco módulos operativos en una arquitectura unificada: gestión de identidades con cuatro niveles jerárquicos y control granular por área, infraestructura PKI con autoridad certificadora interna, gestión documental con firma digital y verificación de integridad, comunicación interna con anuncios y mensajes firmados, y comunicación externa segura con verificación criptográfica de adjuntos. Cada acción sensible queda registrada en una bitácora institucional inmutable que permite reconstruir el histórico completo de operaciones.

3.1.2. Justificación de la elección de stack tecnológico

La elección de PHP 8.2 con Apache y MySQL 8 como stack principal respondió a tres factores. Primero, la compatibilidad nativa con hosting compartido económico como HostGator Plan Negocios, que es la infraestructura realmente accesible para una organización sin fines de lucro como Casa Monarca. Segundo, la madurez de la extensión OpenSSL de PHP, que expone todas las primitivas criptográficas requeridas (RSA, X.509, S/MIME, PKCS#7)

sin necesidad de bibliotecas externas. Tercero, la amplia base de desarrolladores familiarizados con el stack, que facilitará el mantenimiento futuro del sistema por personas distintas al equipo desarrollador original.

Se evaluaron alternativas como Node.js con Express y Python con Django, pero ambas requerirían infraestructura adicional (servidor dedicado, configuración de procesos persistentes, gestión de dependencias más compleja) que excede los recursos de Casa Monarca. La elección de tecnologías ampliamente soportadas en hosting compartido fue una restricción deliberada del proyecto.

3.1.3. Justificación de HostGator frente a AWS u otras plataformas cloud

Una decisión particularmente relevante fue elegir HostGator Plan Negocios como infraestructura de despliegue final, en lugar de servicios cloud como AWS, Google Cloud Platform o Microsoft Azure. Las razones son las siguientes:

Costo previsible y bajo. HostGator Plan Negocios tiene un costo mensual fijo accesible para una organización sin fines de lucro, sin sorpresas por consumo de recursos, transferencia de datos o servicios adicionales. AWS y otras plataformas cloud, aunque ofrecen tiers gratuitos iniciales, generan facturas variables que pueden incrementarse rápidamente conforme crece el uso, lo que es problemático para una organización con presupuesto fijo.

Simplicidad operativa. HostGator proporciona cPanel, una interfaz gráfica intuitiva para administrar archivos, bases de datos, certificados SSL y correos institucionales sin requerir conocimientos avanzados de administración de sistemas. AWS requiere conocimiento técnico considerable para configurar servicios como EC2, RDS, IAM y CloudFront, lo que crearía dependencia con el equipo técnico desarrollador.

Auto-suficiencia institucional. El objetivo del proyecto incluye que Casa Monarca pueda operar el sistema de manera autónoma a futuro. Un hosting compartido permite que personal con conocimientos básicos pueda subir archivos por FTP, importar respaldos de base de datos por phpMyAdmin y gestionar el dominio sin requerir consultoría externa especializada en cloud.

Alcance suficiente. La carga operativa esperada de SIGAM, considerando el tamaño de la organización y su población atendida, está holgadamente dentro de la capacidad de un hosting compartido de gama media. Sobre-aprovisionar con infraestructura cloud para una carga modesta sería un uso ineficiente de los recursos de la organización.

La decisión asume que, si en el futuro Casa Monarca escala su operación al punto de requerir infraestructura más robusta, la arquitectura del sistema permite la migración a un VPS o servicio cloud sin reescritura del código.

3.1.4. Justificación del modelo zero-knowledge para llaves privadas

Una decisión crítica fue establecer que las llaves privadas de los usuarios nunca se almacenan en el servidor. Esta decisión, aunque incrementa la complejidad del frontend al requerir que el usuario suba su archivo `.key` en cada operación de firma, es fundamental por dos razones.

Primero, garantiza el valor probatorio jurídico de las firmas digitales emitidas por los usuarios. Si la llave privada estuviera en el servidor, un usuario podría argumentar legítimamente que cualquier administrador del sistema pudo haber falsificado su firma, lo que invalidaría el no repudio. Segundo, protege a Casa Monarca como organización custodia: en caso de una brecha de seguridad del servidor, las identidades criptográficas de los usuarios permanecen protegidas porque las llaves privadas nunca residieron ahí.

La implementación técnica utiliza la API Web Crypto del navegador, disponible en todos los navegadores modernos. El usuario carga su archivo `.key`, el navegador lo importa al subsistema criptográfico, realiza la operación de firma localmente y envía únicamente el resultado al servidor. Esta arquitectura es la misma que utilizan sistemas como el SAT con la e.firma.

3.1.5. Justificación de la decisión de firma S/MIME con clave del sistema

Para la firma de correos electrónicos salientes se evaluaron tres opciones: firmar con la llave del usuario que dispara el envío, no firmar y simplemente enviar correos planos, o firmar con una llave dedicada del sistema. Se eligió la tercera opción.

Firmar con la llave del usuario habría requerido que el servidor tuviera acceso a esa llave en el momento del envío, contradiciendo el principio de zero-knowledge. No firmar habría dejado los correos sin garantías de integridad ni autenticidad. La opción adoptada, firmar con una llave dedicada del sistema cuyo certificado pertenece a la organización Casa Monarca y no a ningún usuario individual, preserva ambas garantías sin contradecirse con la arquitectura zero-knowledge.

3.1.6. Justificación del modelo de comunicación externa diferenciado

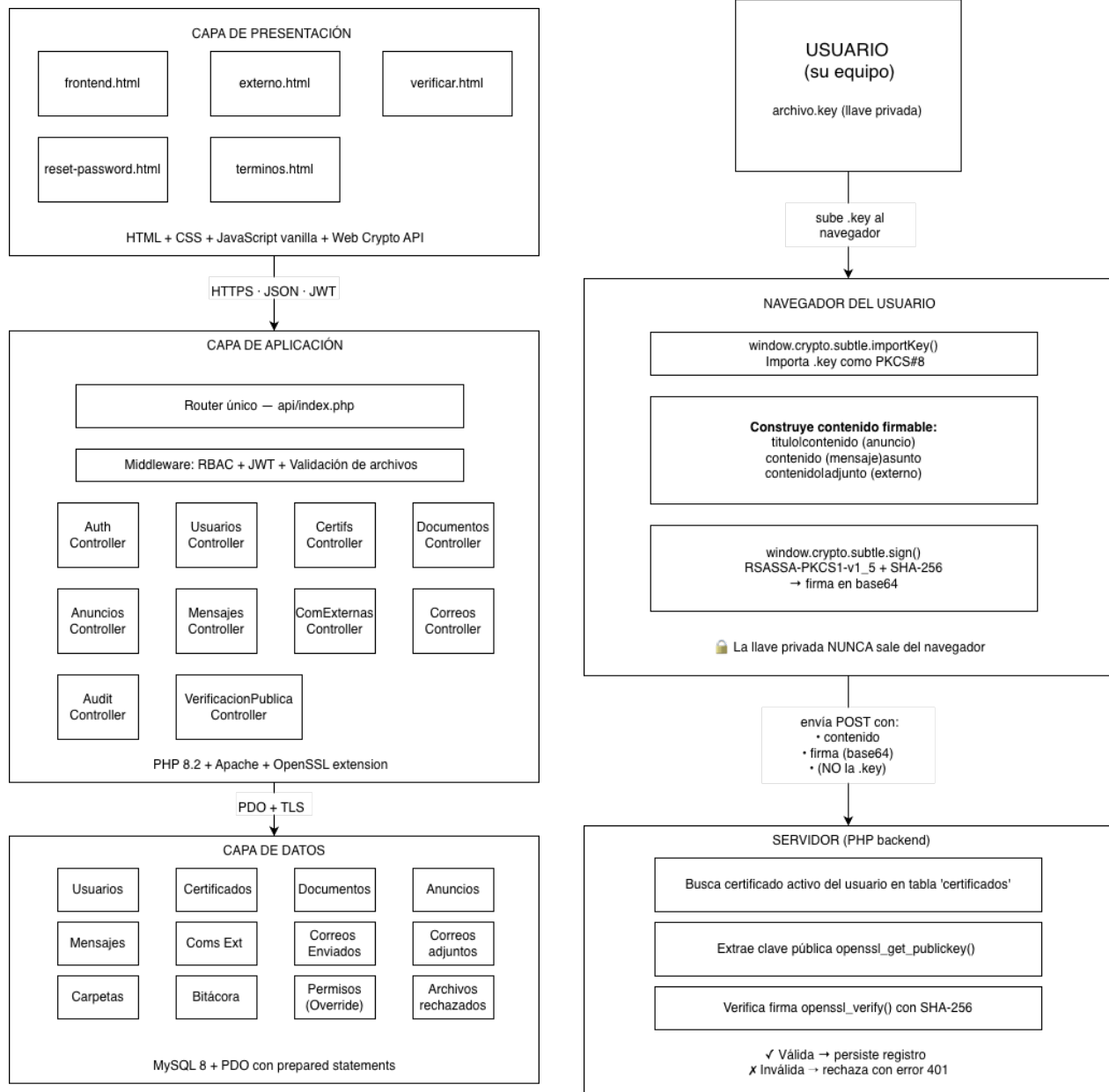
Casa Monarca interactúa con dos tipos de personas externas: aquellas que requieren consultar documentos dentro del sistema (asesores externos, voluntarios de otras organizaciones) y aquellas que únicamente necesitan enviar comunicaciones formales firmadas sin acceder a la plataforma (abogados independientes, contrapartes institucionales). Esta diferencia operativa real motivó el diseño de dos subtipos dentro del nivel 4: externos con acceso a la plataforma y externos exclusivos de comunicación, distinguidos por el flag `solo_comunicacion`.

La decisión evita dos extremos problemáticos: tratar a todos los externos como usuarios completos del sistema (sobrecarga innecesaria para quienes solo necesitan enviar un dictamen) o tratar a todos como contactos externos sin identidad (perdería trazabilidad criptográfica). El modelo diferenciado permite que cada externo reciba exactamente las capacidades que necesita sin más.

3.1.7. Justificación de la validación en tres capas para adjuntos

La validación de archivos adjuntos podría haberse limitado a verificar la extensión, ya que es la solución más sencilla. Sin embargo, una validación tan superficial es trivialmente evadible: cualquier atacante puede renombrar un archivo malicioso. Se adoptó un esquema de defensa en profundidad con tres capas independientes (extensión, tipo MIME detectado, magic bytes binarios) y, para archivos Office, una cuarta capa de inspección de la estructura ZIP

3.1.8. Diagrama de la solución



(a) Arquitectura de tres capas.

(b) Flujo zero-knowledge.

Figura 1: Diagramas de la propuesta de solución.³

³Elaboración propia.

3.2. Desarrollo de componentes: Backend

El backend de SIGAM se compone de un conjunto de controladores especializados que operan sobre la base de datos a través de PDO, coordinados por un router único y protegidos por middleware centralizado. A continuación se describen los componentes más relevantes.

3.2.1. Router único y despacho de peticiones

El archivo `api/index.php` actúa como router único del backend. Recibe todas las peticiones HTTP que llegan a `/api/`, extrae el recurso y la acción solicitados de la URL (formato `/api/{recurso}/{acción}`), incluye el controlador correspondiente y despacha al método adecuado. Este patrón centraliza la configuración de CORS, headers de seguridad, manejo de errores no capturados y logging básico de peticiones, evitando duplicación de código en cada controlador.

3.2.2. Middleware RBAC

El módulo `middleware/rbac.php` centraliza la lógica de autorización del sistema. Expone la función `verificarAcceso($nivel_minimo, $area, $permiso)` que se invoca al inicio de cada método de controlador que requiere autenticación. Esta función valida tres condiciones: que el token JWT recibido sea válido y no esté expirado, que el nivel del usuario sea igual o superior al requerido, y que el usuario tenga el permiso específico solicitado (`r`, `w`, `x` o `e`). Adicionalmente, calcula los permisos efectivos del usuario combinando los permisos base de su nivel con los overrides individuales registrados en la tabla `permisos_overrides`, permitiendo ajustes granulares sin necesidad de modificar la estructura de niveles.

3.2.3. Controlador de autenticación

El `AuthController` maneja el flujo de login y emisión de tokens JWT. Recibe email y contraseña, verifica la contraseña contra el hash bcrypt almacenado en la tabla `usuarios` mediante `password_verify()`, y si es válida emite un JWT firmado con HS256 que incluye en su payload el identificador del usuario, su nivel, su área y un timestamp de expiración. El token se devuelve al cliente, que lo incluye en el header `Authorization: Bearer ...` de las peticiones subsecuentes.

3.2.4. Controlador de certificados

El `CertificadosController` gestiona el ciclo de vida completo de los certificados digitales. La emisión utiliza `openssl_pkey_new` para generar un par de llaves RSA-2048,

`openssl_csr_new` para construir el CSR con los datos del usuario, y `openssl_csr_sign` para firmar el certificado con la CA institucional. El certificado resultante se exporta en formato PEM y se almacena en la tabla `certificados`, mientras que la llave privada se coloca temporalmente en un directorio de descarga con un link único de expiración de 24 horas. La renovación, revocación y reemisión siguen patrones similares, registrando cada operación en la bitácora.

3.2.5. Controlador de documentos

El `DocumentosController` expone los métodos para crear, listar, ver, firmar, mover y autorizar documentos. La creación recibe el archivo en multipart, lo codifica en base64 antes de almacenarlo en la columna `contenido_cifrado`, calcula su hash SHA-256 y lo persiste como evidencia de integridad. La firma de un documento aplica `openssl_verify` sobre la cadena `texto + '|' + contenido_cifrado` usando la llave pública del autor extraída de su certificado activo, garantizando que solo el dueño de la llave privada pudo haber producido esa firma.

3.2.6. Helper de mensajería

El módulo `middleware/mensajeria.php` centraliza las reglas de comunicación interna. Expone las funciones `puedeMandarMensajeA($emisor, $receptor)` y `puedeMandarAnuncio($emisor)`, que codifican la lógica de quién puede comunicarse con quién según nivel y área. Esta separación permite que tanto el controlador de mensajes como el de anuncios consulten las mismas reglas, garantizando consistencia.

3.2.7. Helper de validación de archivos

El módulo `middleware/validacion_archivos.php` expone la clase `ValidacionArchivos` con el método estático principal `validar($archivo)`. Aplica las tres capas de validación descritas en la sección de Marco de Referencia (extensión, MIME, magic bytes), la validación heurística para texto plano, la detección de macros en archivos Office mediante inspección de la estructura ZIP interna, y el cálculo del hash SHA-256 final. Cada rechazo se registra en la tabla `archivos_rechazados` junto con metadatos forenses para auditoría posterior.

3.2.8. Mailer con firma S/MIME

El componente `config/mail.php` provee la clase `Mailer` responsable del envío de correos electrónicos. El método `send()` firma cada correo saliente con S/MIME aplicando

`openssl_pkcs7_sign()` sobre el cuerpo del mensaje con el certificado dedicado del sistema, produciendo un adjunto `smime.p7s` conforme al RFC 5751. El método extendido `sendConAdjuntos()` construye un mensaje multipart MIME que incluye los archivos adjuntos antes de aplicar la firma, garantizando que la firma cubra el mensaje completo incluyendo los anexos.

3.2.9. Tareas programadas

El sistema incluye dos scripts cron diseñados para ejecutarse mediante `cron/avisos_expiracion.php` que revisa diariamente los certificados próximos a expirar y envía avisos a los titulares, y `cron/verificar_hashes_adjuntos.php` que recalcula los hashes SHA-256 de todos los adjuntos almacenados y detecta manipulación post-recepción. Cualquier discrepancia se registra en la bitácora con la acción `INTEGRIDAD_ADJUNTO_COMPROMETIDA` y dispara una alerta firmada al administrador.

3.2.10. Bitácora institucional

La tabla `bitacora` actúa como log inmutable de operaciones sensibles. Cada controlador, antes de retornar una respuesta exitosa de una operación que modifique datos, inserta un registro con el identificador del usuario, la acción ejecutada, la tabla afectada, el identificador del registro modificado y el timestamp. Esta inserción es parte de la misma transacción que la operación principal, garantizando que ninguna operación pueda completarse sin dejar trazabilidad. La bitácora es consultable desde el frontend del administrador mediante el `AuditController` con filtros por acción, usuario, fecha y tabla afectada.

3.3. Desarrollo de componentes — Frontend

El frontend de SIGAM se construyó como una aplicación de página única (SPA) en HTML, CSS y JavaScript vanilla, sin frameworks externos. Esta decisión obedece a tres factores: la simplicidad de mantenimiento posterior por parte de Casa Monarca, la ausencia de dependencias que requieran un proceso de build, y la compatibilidad directa con hosting compartido sin necesidad de servidor Node.js para servir archivos estáticos. A continuación se describen los componentes frontend más relevantes.

3.3.1. Frontend principal: `frontend.html`

La interfaz principal es un único archivo HTML autocontenido que incluye estilos y lógica completa de la SPA. Al cargar, ejecuta una función `init()` que valida el token JWT contra

el endpoint `/api/auth/me` y, si es válido, construye dinámicamente el sidebar y el área principal según el nivel del usuario. La navegación entre secciones se hace mediante una función `navigateTo(seccion)` que mapea identificadores de sección a funciones `render` específicas, evitando recargas de página.

El estado del usuario autenticado se mantiene en una variable global `SESSION` que contiene su identificador, nombre, email, nivel, área, matrícula y flags relevantes. Esta variable se consulta desde cada función `render` para personalizar la interfaz y restringir acciones según los permisos del usuario.

3.3.2. Sidebar dinámico

La función `buildSidebar()` construye la barra lateral según el nivel del usuario, mostrando únicamente las secciones a las que tiene acceso. Por ejemplo, el administrador ve secciones como *Colaboradores*, *Certificados*, *Auditoría* y *Comunicaciones externas*, mientras que un colaborador nivel 4 únicamente ve *Documentos autorizados* y *Mensajes*. Esta lógica de presentación complementa la validación de permisos del backend, mejorando la experiencia del usuario al no mostrar opciones que no podrá ejecutar.

3.3.3. Wrapper de API y manejo de tokens

Un objeto `API` encapsula las llamadas al backend con los métodos `get(path)` y `post(path, body)`. Cada petición incluye automáticamente el token JWT en el header `Authorization`, maneja respuestas JSON, detecta errores de autenticación expirada y redirige al login cuando es necesario. La constante `BASE_URL` se define como `window.location.origin`, lo que permite que el sistema funcione correctamente tanto en `localhost` durante desarrollo como en el dominio de producción sin modificar código.

3.3.4. Firma digital local con Web Crypto API

La función `importarLlavePEM(pemText)` es uno de los componentes más críticos del frontend. Recibe el contenido del archivo `.key` del usuario en formato PEM, le quita los delimitadores y espacios, decodifica el base64 a binario y lo importa al subsistema criptográfico del navegador con `crypto.subtle.importKey()` bajo el algoritmo `RSASSA-PKCS1-v1_5` con hash `SHA-256`. La función devuelve un objeto `CryptoKey` no exportable que únicamente puede usarse para firmar dentro de la misma sesión del navegador.

Una vez importada la llave, la firma se aplica con `crypto.subtle.sign()`. El resultado binario se codifica en base64 y se envía al backend junto con el contenido firmado. Esta

secuencia se reutiliza en todos los flujos que requieren firma digital: documentos, anuncios, mensajes y comunicaciones externas.

3.3.5. Modales reutilizables

El frontend utiliza un patrón consistente de modales para acciones que requieren capturar información del usuario. Existen funciones genéricas `openModal(id)` y `closeModal(id)` que aplican o quitan la clase `hidden` sobre el contenedor del modal, junto con funciones específicas para cada modal: `openModalNuevoMensaje()`, `abrirModalNuevoCorreo()`, `configurarModalAnuncio()`, entre otras. Cada función específica prepara el modal antes de mostrarlo: limpia campos previos, carga datos asíncronos (como la lista de contactos disponibles) y aplica restricciones según el contexto.

3.3.6. Páginas públicas dedicadas

Cinco páginas HTML independientes atienden flujos que no requieren autenticación previa:

verificar.html permite a cualquier destinatario externo subir un archivo `.eml` para validar la firma S/MIME contra el certificado del sistema. Implementa una interfaz minimalista con un input de archivo, un botón de verificación y un área de resultado que muestra el estado de la firma, el remitente, asunto y fecha extraídos del correo.

externo.html es la página de entrada para colaboradores externos exclusivos de comunicación. Recibe el identificador del externo por query string (`?id=...`), carga sus datos desde `/api/externos/info` para personalizar la bienvenida, presenta un formulario con asunto, contenido y adjunto opcional, y exige la llave privada para firmar localmente el envío. El flujo de firma reutiliza la misma lógica de Web Crypto API descrita anteriormente.

reset-password.html maneja el flujo de restablecimiento de contraseña iniciado desde el login. Recibe un token único por query string, valida que sea reciente y no esté usado, y permite al usuario establecer una nueva contraseña que se hashea con `bcrypt` en el backend.

terminos.html contiene el aviso de términos y condiciones de uso del sistema, accesible desde el sidebar y desde correos institucionales como referencia legal.

seed.php es un script de inicialización ejecutable una sola vez durante la instalación, que crea el usuario administrador inicial y su cuenta espejo, devolviendo las credenciales temporales y el link de descarga de los certificados bootstrap.

3.3.7. Componentes visuales y feedback al usuario

El frontend incorpora un conjunto de utilidades visuales reutilizables: la función `toast(mensaje, esError)` muestra notificaciones temporales en la esquina inferior; la función `setLoading(boton, activo, textoOriginal)` desactiva botones durante peticiones asíncronas mostrando un spinner; y la función `showErr(elemento, mensaje)` despliega errores inline en los formularios. Estos componentes garantizan que el usuario reciba retroalimentación clara en cada operación, especialmente importante en flujos criptográficos donde una operación puede fallar por múltiples razones (firma inválida, certificado expirado, archivo corrupto).

3.3.8. Visualización de permisos con iconos

Los permisos del modelo RBAC se representan visualmente en la interfaz mediante iconos intuitivos: un símbolo de libro para lectura, un lápiz para escritura, un candado abierto para autorización y una silueta de persona para administración. Esta decisión, tomada después de iteraciones con el diseño original que mostraba las letras `r/w/x/e`, mejora significativamente la comprensión del modelo de permisos por parte de usuarios no técnicos, especialmente durante la edición de overrides individuales por parte del administrador.

3.4. Conexión de los componentes

La comunicación entre las tres capas del sistema sigue patrones consistentes que se aplican uniformemente en cada flujo. Esta subsección describe cómo se conectan el frontend, el backend y la base de datos, y cómo se integran las capas adicionales de seguridad y verificación criptográfica.

3.4.1. Comunicación entre frontend y backend

Toda la comunicación entre la SPA y el backend ocurre mediante peticiones HTTP REST que transportan datos en formato JSON. El frontend utiliza la función nativa `fetch` del navegador para enviar peticiones, encapsulada en el wrapper `API.get()` y `API.post()`. Cada petición incluye automáticamente dos elementos: el header `Content-Type: application/json` cuando hay cuerpo, y el header `Authorization: Bearer {token}` cuando la petición requiere autenticación.

El backend recibe la petición en el router único `api/index.php`, extrae el recurso y la acción de la URL, valida el token JWT, despacha al método correspondiente del controlador y devuelve la respuesta como JSON. El frontend procesa esa respuesta, actualiza su estado interno y refleja los cambios en la interfaz mediante las funciones de render correspondientes.

3.4.2. Autenticación y manejo de sesiones

El flujo de autenticación inicia con un POST a `/api/auth/login` con email y contraseña. El backend valida las credenciales contra el hash bcrypt almacenado, y si son correctas devuelve un JWT firmado con HS256 que incluye en su payload el identificador del usuario, su nivel y un timestamp de expiración. El frontend almacena este token en `localStorage` bajo la clave `token` y lo recupera en cada petición subsecuente.

Cuando el token expira o es inválido, el backend devuelve un código HTTP 401 que el wrapper de API detecta automáticamente, limpia el almacenamiento local y redirige al usuario a la pantalla de login. Esto garantiza que ninguna operación pueda ejecutarse con credenciales caducadas.

3.4.3. Comunicación entre backend y base de datos

El backend accede a MySQL exclusivamente a través de PDO (PHP Data Objects), una capa de abstracción que provee soporte unificado para múltiples bases de datos y, más importante, soporte nativo para prepared statements. Cada consulta SQL del sistema se construye con placeholders parametrizados (`?` o `:nombre`) y los valores se pasan por separado al método `execute()`, lo que previene inyección SQL por diseño sin requerir sanitización manual.

La conexión a la base se establece mediante la función `getDB()` definida en `config/database.php`, que aplica el patrón singleton para reutilizar una única conexión durante toda la ejecución de la petición. Las credenciales de conexión se leen del archivo `.env`, manteniendo la configuración separada del código.

3.4.4. Integración con el subsistema criptográfico del navegador

Los flujos que requieren firma digital establecen una conexión especial entre frontend y navegador. Cuando el usuario sube su archivo `.key`, el navegador lo procesa en memoria mediante la Web Crypto API sin enviarlo al backend. El proceso ocurre en cuatro pasos:

1. El navegador lee el archivo con la API `File` y obtiene su contenido como texto PEM.
2. La función `importarLlavePEM` extrae el contenido base64, lo decodifica a binario y lo importa al subsistema criptográfico con `crypto.subtle.importKey` bajo el algoritmo RSASSA-PKCS1-v1_5 con SHA-256.
3. El frontend construye la cadena a firmar concatenando los campos relevantes del formulario, la codifica como bytes con `TextEncoder` y aplica `crypto.subtle.sign` con la llave importada.

4. La firma resultante se codifica en base64 y se envía al backend junto con el contenido firmado.

El backend recibe la petición, busca el certificado activo del usuario en la base de datos, extrae la llave pública con `openssl_get_publickey` y verifica la firma con `openssl_verify` usando el mismo algoritmo. Esta simetría entre cliente y servidor garantiza compatibilidad criptográfica sin requerir bibliotecas adicionales en ningún extremo.

3.4.5. Integración con el servidor SMTP para correos firmados

El componente Mailer del backend se conecta al servidor SMTP configurado en el archivo `.env` (durante desarrollo, Mailtrap; en producción, el servidor institucional). El proceso de envío incluye una capa adicional de firma S/MIME que ocurre antes de la entrega al SMTP:

1. El controlador que dispara el correo construye el cuerpo HTML del mensaje usando las plantillas de `config/templates.php`.
2. El método `Mailer::send()` escribe el cuerpo del mensaje en un archivo temporal junto con los headers principales.
3. Aplica `openssl_pkcs7_sign()` con el certificado y llave del sistema, produciendo un mensaje firmado en formato multipart/signed conforme al RFC 5751.
4. Establece una conexión TCP con el servidor SMTP mediante `fsockopen`, ejecuta el handshake autenticado y transmite el mensaje firmado.

Esta integración garantiza que todo correo saliente del sistema lleve firma criptográfica institucional sin requerir configuración adicional por parte del usuario que disparó el envío.

3.4.6. Tareas programadas mediante cron

Los scripts de tareas programadas se conectan al sistema desde fuera del flujo HTTP. El crontab del servidor invoca diariamente `cron/avisos_expiracion.php` mediante el comando `docker compose exec` en entorno de desarrollo, o directamente mediante el binario de PHP en producción. Estos scripts utilizan las mismas funciones de conexión a base de datos y de envío de correos que los controladores HTTP, lo que mantiene la coherencia de la lógica sin duplicación de código.

3.4.7. Flujo completo de un caso ilustrativo

Para ilustrar la conexión integral de los componentes, considérese el flujo de publicación de un anuncio firmado por un coordinador:

1. El coordinador inicia sesión, el frontend obtiene el token JWT y lo almacena en `localStorage`.
2. El coordinador navega a la sección de Anuncios y abre el modal de nuevo anuncio.
3. El frontend ajusta el modal según el nivel del usuario, mostrando únicamente opciones de alcance permitidas para coordinadores.
4. El coordinador completa el título, contenido y sube su archivo `.key`.
5. El navegador importa la llave con Web Crypto API, firma la cadena `título|contenido` y obtiene la firma en base64.
6. El frontend envía POST a `/api/anuncios/crear` con el contenido y la firma, incluyendo el token JWT en el header.
7. El router despacha al `AnunciosController`, el middleware RBAC valida nivel y permiso, el controlador verifica la firma contra el certificado activo del coordinador.
8. Si la firma es válida, se inserta el anuncio en la tabla `anuncios` con la firma incluida, se identifican los destinatarios según el alcance, y se registra la acción en la bitácora.
9. Para cada destinatario, el Mailer construye el correo de notificación, lo firma con S/MIME usando el certificado del sistema y lo envía por SMTP.
10. El backend devuelve respuesta JSON al frontend con el ID del anuncio creado y el conteo de destinatarios notificados.
11. El frontend cierra el modal, muestra una notificación toast de éxito y recarga la lista de anuncios.

Este flujo único de extremo a extremo demuestra cómo los componentes individuales colaboran para producir una operación firmada criptográficamente verificable, con trazabilidad completa en bitácora y notificación firmada a los destinatarios.

4. Resultados

4.1. Resultados del Backend

El backend desarrollado consta de once controladores especializados, tres archivos de middleware, dos scripts de tareas programadas, un mailer con soporte S/MIME y dos archivos de configuración. La totalidad del backend implementa el patrón de prepared statements en cada interacción con la base de datos, sin excepciones, lo que garantiza prevención de inyección SQL por diseño en la totalidad del código.

Los principales logros del backend son los siguientes. La función `verificarAcceso` del middleware RBAC se invoca consistentemente al inicio de cada método de controlador que requiere autenticación, centralizando la lógica de autorización en un único punto del código. La capa criptográfica delega todas las operaciones a la extensión OpenSSL nativa de PHP, evitando dependencias externas que pudieran requerir actualizaciones de seguridad independientes. La emisión de certificados, la verificación de firmas digitales y la aplicación de firmas S/MIME funcionan correctamente con los parámetros estándar de la industria (RSA-2048, SHA-256, X.509 v3, PKCS#7 detached).

La verificación de integridad de adjuntos opera de manera completamente desatendida mediante el cron diario, registrando cualquier discrepancia en la bitácora institucional y enviando alertas firmadas a los administradores. Durante las pruebas de regresión, modificaciones intencionales sobre el campo `adjunto_contenido` fueron detectadas correctamente por la verificación, validando el funcionamiento del mecanismo.

4.2. Resultados del Frontend

El frontend resultante consta de un archivo principal `frontend.html` que implementa toda la SPA del sistema, cinco páginas públicas independientes que atienden flujos sin autenticación, y un script de inicialización `seed.php`. La aplicación funciona sin recargas de página tras el login inicial, con navegación fluida entre secciones mediante funciones de render asincrónicas.

Los principales logros del frontend son los siguientes. La interfaz se adapta dinámicamente al nivel del usuario autenticado, mostrando únicamente las secciones y acciones que tiene permitidas. El sidebar se construye en tiempo de carga según el nivel detectado en `SESSION.nivel`, y los modales de creación o edición ajustan sus campos visibles según el contexto. La validación de la sesión ocurre contra el endpoint `/api/auth/me` al cargar la aplicación, garantizando que tokens expirados o inválidos no permitan acceso a la interfaz.

La integración con la Web Crypto API funciona correctamente en los navegadores mo-

ernos probados (Chrome 147, Firefox 130, Safari 17, Edge 132). La importación de llaves privadas en formato PKCS#8 y la generación de firmas con RSASSA-PKCS1-v1_5 producen resultados compatibles con la verificación posterior en OpenSSL del backend, validando la simetría criptográfica entre cliente y servidor.

Las páginas públicas dedicadas (`verificar.html`, `externo.html`, `reset-password.html`, `terminos.html`) operan correctamente sin requerir autenticación, atendiendo los flujos para destinatarios externos sin acceso a la plataforma principal.

4.3. Resultados de la solución completa

La integración de backend y frontend produce un sistema funcional completo que opera de extremo a extremo. Cada uno de los cinco módulos planteados originalmente quedó implementado y verificado: gestión de identidades con cuatro niveles jerárquicos y matrículas institucionales, infraestructura PKI con autoridad certificadora interna operando ciclo completo de certificados, gestión documental con firma digital y verificación periódica de integridad, comunicación interna con anuncios firmados obligatoriamente y mensajes con firma opcional, y comunicación externa segura con verificación criptográfica de adjuntos en tres capas.

La bitácora institucional registra correctamente todas las operaciones sensibles del sistema. Durante las pruebas, se verificó que las acciones como creación de usuarios, emisión de certificados, firma de documentos, publicación de anuncios, envío de mensajes y recepción de comunicaciones externas quedan registradas con timestamp, identificador de usuario y registro afectado.

La plataforma se desplegó exitosamente en entorno local mediante Docker Compose con tres contenedores (PHP-Apache, MySQL, phpMyAdmin), y se verificó que la configuración es compatible con el despliegue planeado en HostGator Plan Negocios. El sistema responde con tiempos aceptables para el volumen esperado de operaciones de Casa Monarca, considerando que la base de usuarios y el tráfico esperado están holgadamente dentro de la capacidad del hosting compartido seleccionado.

4.4. Casos de uso y tabla de validaciones

A continuación se presentan los casos de uso principales del sistema, junto con los criterios de validación verificados y las evidencias correspondientes. Las evidencias detalladas se encuentran disponibles en el repositorio público del proyecto.

Tabla 1: Casos de uso y tabla de validaciones del sistema SIGAM.⁴

Caso de uso	Criterio de validación	Evidencias
Login del administrador y validación de sesión	El JWT emitido contiene nivel y área correctos; sesiones expiradas son rechazadas con HTTP 401	Repositorio: /evidencias/login
Creación y activación de colaborador	El colaborador queda inactivo hasta ser activado; el correo de credenciales se envía firmado con S/MIME al activarse	Repositorio: /evidencias/usuarios
Emisión de certificado digital	El certificado X.509 generado contiene los datos del titular, llave pública RSA-2048 y vigencia de 4 años	Repositorio: /evidencias/certificados
Firma digital de documento con zero-knowledge	La llave privada no aparece en logs del servidor; la firma generada en navegador valida correctamente en backend	Repositorio: /evidencias/firma_documento
Publicación de anuncio institucional firmado	Anuncios sin firma son rechazados con HTTP 400; anuncios con firma válida se publican y notifican por correo	Repositorio: /evidencias/anuncios
Restricción de comunicación por nivel y área	Nivel 4 no puede iniciar conversación con usuarios fuera de su área; el backend rechaza intentos manipulados con HTTP 403	Repositorio: /evidencias/mensajes
Recepción de comunicación externa firmada	La firma del externo es verificada contra su certificado activo; firmas inválidas son rechazadas con HTTP 401	Repositorio: /evidencias/com_externa

Caso de uso	Criterio de validación	Evidencias
Validación de adjuntos en tres capas	Archivos con extensión renombrada (ej. .exe a .pdf) son detectados por magic bytes y rechazados; los rechazos quedan en cuarentena	Repositorio: /evidencias/validacion_adjuntos
Detección de macros en archivos Office	Archivos DOCX/XLSX/PPTX que contienen vbaProject.bin son rechazados con mensaje específico	Repositorio: /evidencias/macros_office
Verificación pública de correo S/MIME	Cualquier destinatario externo puede subir un .eml y obtener verificación criptográfica de la firma	Repositorio: /evidencias/verificador
Envío de correo institucional con adjuntos	Correos enviados por admin o coordinadores llegan firmados con S/MIME y con sus adjuntos cubiertos por la firma	Repositorio: /evidencias/correos_institucionales
Recuperación administrativa mediante cuenta espejo	La cuenta espejo no puede ejecutar otras operaciones administrativas; la recuperación queda registrada en bitácora	Repositorio: /evidencias/recuperacion
Verificación periódica de integridad de adjuntos	El cron diario detecta modificaciones manuales en adjuntos almacenados y envía alerta al administrador	Repositorio: /evidencias/integridad

⁴Elaboración propia. Las evidencias detalladas (capturas de pantalla, videos cortos y logs) se encuentran en el repositorio público del proyecto.

5. Conclusiones y trabajo futuro

5.1. Conclusiones

El proyecto SIGAM cumplió con los objetivos planteados al inicio del reto, entregando una plataforma web institucional funcional que resuelve la problemática original de gestión manual sin garantías criptográficas sobre información sensible. Los esquemas implementados (RSA-2048 con SHA-256, X.509, bcrypt, S/MIME, JWT) corresponden a estándares maduros de la industria, priorizando soluciones probadas sobre alternativas más modernas pero menos accesibles para una organización sin fines de lucro.

La decisión arquitectónica más relevante fue la adopción del principio de zero-knowledge sobre las llaves privadas de los usuarios. Esta decisión, aunque incrementó la complejidad del frontend, fue fundamental para preservar el valor probatorio de las firmas digitales y proteger las identidades criptográficas frente a eventuales brechas del servidor. La defensa en profundidad aplicada en la validación de adjuntos y el modelo RBAC extendido con dimensión de área y overrides individuales completan una postura de seguridad razonable considerando el contexto y los recursos disponibles.

Algunas decisiones quedaron como limitaciones conscientes y documentadas, particularmente la ausencia de cifrado a nivel motor en la base de datos. Estas limitaciones son honestas y defendibles en el alcance del proyecto académico, y su atención se incorpora al roadmap de la versión 2.

5.2. Trabajo futuro

Las siguientes mejoras se identifican como roadmap para una versión 2 del sistema, ordenadas por prioridad:

- **Cifrado de la base de datos a nivel motor (TDE)** mediante AES-256 sobre los archivos físicos del tablespace.
- **Cifrado en tránsito** de la conexión PHP-MySQL mediante TLS.
- **Integración con motor antivirus** (ClamAV) para escaneo de adjuntos.
- **Autenticación de doble factor** con TOTP para usuarios de niveles 1 y 2.
- **Migración del almacenamiento de archivos** a object storage especializado.
- **Módulo de reportes y estadísticas operativas** a partir de la bitácora institucional.
- **Implementación formal de derechos ARCO** conforme a la LFPDPPP.

- **Plan formal de respuesta ante incidentes** y capacitación periódica del personal.

La implementación progresiva de estas mejoras posicionaría a SIGAM como una plataforma institucional madura, equiparable en garantías de seguridad a soluciones empresariales especializadas.

Anexo: Repositorio del proyecto

El código fuente completo del proyecto, junto con la documentación técnica, el manual de usuario, las evidencias de los casos de uso y los reportes ejecutivo y técnico, se encuentra disponible en el repositorio público de GitHub:

<https://github.com/carolavazquez/MA2006B>

El repositorio se distribuye bajo licencia MIT y contiene la estructura organizada en código fuente (`src/`), documentación (`docs/`), evidencias por caso de uso y el archivo `README.md` con las instrucciones de instalación, configuración y uso del sistema.

Referencias

- [1] Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.
- [2] Barker, E. (2020). *Recommendation for Key Management: Part 1 – General* (NIST SP 800-57 Part 1 Rev. 5). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-57pt1r5>
- [3] National Institute of Standards and Technology. (2015). *Secure Hash Standard (SHS)* (FIPS PUB 180-4). <https://doi.org/10.6028/NIST.FIPS.180-4>
- [4] Provos, N., & Mazières, D. (1999). A Future-Adaptable Password Scheme. *Proceedings of the USENIX Annual Technical Conference*.
- [5] Cooper, D., Santesson, S., Farrell, S., Boeyen, S., Housley, R., & Polk, W. (2008). *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile* (RFC 5280). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc5280>

- [6] Ramsdell, B., & Turner, S. (2010). *Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 3.2 Message Specification* (RFC 5751). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc5751>
- [7] Housley, R. (2009). *Cryptographic Message Syntax (CMS)* (RFC 5652). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc5652>
- [8] Sandhu, R., Ferraiolo, D., & Kuhn, R. (2000). The NIST Model for Role-Based Access Control: Towards a Unified Standard. *Proceedings of the 5th ACM Workshop on Role-Based Access Control*.
- [9] Microsoft Learn. (s.f.). *Microsoft Entra ID documentation*. <https://learn.microsoft.com/en-us/entra/identity/>
- [10] Microsoft Learn. (s.f.). *Azure Key Vault Overview*. <https://learn.microsoft.com/en-us/azure/key-vault/general/overview>
- [11] Microsoft Learn. (s.f.). *Transparent Data Encryption (TDE) – SQL Server*. <https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/transparent-data-encryption>
- [12] National Institute of Standards and Technology. (2023). *Advanced Encryption Standard (AES)* (FIPS PUB 197, Update 1). <https://doi.org/10.6028/NIST.FIPS.197-upd1>